JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Interface MulticastChannel

All Superinterfaces:

`AutoCloseable`, `Channel`, `Closeable`, `NetworkChannel`

All Known Implementing Classes:

`DatagramChannel`

```
public interface MulticastChannel
extends NetworkChannel
```

A network channel that supports Internet Protocol (IP) multicasting.

IP multicasting is the transmission of IP datagrams to members of a *group* that is zero or more hosts identified by a single destination address.

In the case of a channel to an `IPv4` socket, the underlying operating system supports *RFC 2236: Internet Group Management Protocol, Version 2 (IGMPv2)*. It may optionally support source filtering as specified by *RFC 3376: Internet Group Management Protocol, Version 3 (IGMPv3)*. For channels to an `IPv6` socket, the equivalent standards are *RFC 2710: Multicast Listener Discovery (MLD) for IPv6* and *RFC 3810: Multicast Listener Discovery Version 2 (MLDv2) for IPv6*.

The `join(InetAddress,NetworkInterface)` method is used to join a group and receive all multicast datagrams sent to the group. A channel may join several multicast groups and may join the same group on several `interfaces`. Membership is dropped by invoking the `drop` method on the returned `MembershipKey`. If the underlying platform supports source filtering then the `block` and `unblock` methods can be used to block or unblock multicast datagrams from particular source addresses.

The `join(InetAddress,NetworkInterface,InetAddress)` method is used to begin receiving datagrams sent to a group whose source address matches a given source address. This method throws `UnsupportedOperationException` if the underlying

platform does not support source filtering. Membership is *cumulative* and this method may be invoked again with the same group and interface to allow receiving datagrams from other source addresses. The method returns a `MembershipKey` that represents membership to receive datagrams from the given source address. Invoking the key's `drop` method drops membership so that datagrams from the source address can no longer be received.

Platform dependencies

The multicast implementation is intended to map directly to the native multicasting facility. Consequently, the following items should be considered when developing an application that receives IP multicast datagrams:

1. The creation of the channel should specify the `ProtocolFamily` that corresponds to the address type of the multicast groups that the channel will join. There is no guarantee that a channel to a socket in one protocol family can join and receive multicast datagrams when the address of the multicast group corresponds to another protocol family. For example, it is implementation specific if a channel to an `IPv6` socket can join an `IPv4` multicast group and receive multicast datagrams sent to the group.

2. The channel's socket should be bound to the `wildcard` address. If the socket is bound to a specific address, rather than the wildcard address then it is implementation specific if multicast datagrams are received by the socket.

3. The `SO_REUSEADDR` option should be enabled prior to `binding` the socket. This is required to allow multiple members of the group to bind to the same address.

Usage Example:

```
// join multicast group on this interface, and also use this
// interface for outgoing multicast datagrams
NetworkInterface ni = NetworkInterface.getByName("hme0");

DatagramChannel dc = DatagramChannel.open(StandardProtocolFamily.INET)
    .setOption(StandardSocketOptions.SO_REUSEADDR, true)
    .bind(new InetSocketAddress(5000))
    .setOption(StandardSocketOptions.IP_MULTICAST_IF, ni);

InetAddress group = InetAddress.getByName("225.4.5.6");

MembershipKey key = dc.join(group, ni);
```

Since:

1.7

Method Summary

Abstract Methods

Modifier and Type	Method	Description
void	close()	Closes this channel.
MembershipKey	join(InetAddress group, NetworkInterface interf)	Joins a multicast group to begin receiving all datagrams sent to the group, returning a membership key.
MembershipKey	join(InetAddress group, NetworkInterface interf, InetAddress source)	Joins a multicast group to begin receiving datagrams sent to the group from a given source address.

Methods inherited from interface java.nio.channels.NetworkChannel

bind, getLocalAddress, getOption, setOption, supportedOptions

Methods inherited from interface java.nio.channels.Channel

isOpen

Method Detail

close

```
void close()
    throws IOException
```

Closes this channel.

If the channel is a member of a multicast group then the membership is dropped. Upon return, the membership-key will be invalid.

This method otherwise behaves exactly as specified by the Channel interface.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Channel

Specified by:

close in interface Closeable

Throws:

IOException - If an I/O error occurs

join

```
MembershipKey join(InetAddress group,
                   NetworkInterface interf)
            throws IOException
```

Joins a multicast group to begin receiving all datagrams sent to the group, returning a membership key.

If this channel is currently a member of the group on the given interface to receive all datagrams then the membership key, representing that membership, is returned. Otherwise this channel joins the group and the resulting new membership key is returned. The resulting membership key is not source-specific.

A multicast channel may join several multicast groups, including the same group on more than one interface. An implementation may impose a limit on the number of groups that may be joined at the same time.

Parameters:

group - The multicast address to join

interf - The network interface on which to join the group

Returns:

The membership key

Throws:

IllegalArgumentException - If the group parameter is not a multicast address, or the group parameter is an address type that is not supported by this channel

IllegalStateException - If the channel already has source-specific membership of the group on the interface

UnsupportedOperationException - If the channel's socket is not an Internet Protocol socket

ClosedChannelException - If this channel is closed

IOException - If an I/O error occurs

SecurityException - If a security manager is set, and its checkMulticast method denies access to the multiast group

join

```
MembershipKey join(InetAddress group,
                   NetworkInterface interf,
                   InetAddress source)
            throws IOException
```

Joins a multicast group to begin receiving datagrams sent to the group from a given source address.

If this channel is currently a member of the group on the given interface to receive datagrams from the given source address then the membership key, representing that membership, is returned. Otherwise this channel joins the group and the resulting new membership key is returned. The resulting membership key is source-specific.

Membership is *cumulative* and this method may be invoked again with the same group and interface to allow receiving datagrams sent by other source addresses to the group.

Parameters:

group - The multicast address to join

interf - The network interface on which to join the group

source - The source address

Returns:

The membership key

Throws:

IllegalArgumentException - If the group parameter is not a multicast address, the source parameter is not a unicast address, the group parameter is an address type that is not supported by this channel, or the source parameter is not the same address type as the group

IllegalStateException - If the channel is currently a member of the group on the given interface to receive all datagrams

UnsupportedOperationException - If the channel's socket is not an Internet Protocol socket or source filtering is not supported

ClosedChannelException - If this channel is closed

IOException - If an I/O error occurs

C

```
SecurityException - If a security manager is set, and its
checkMulticast method denies access to the multiast group
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.